Does Deterministic Generate–Validate–Repair Reduce Unsafe LLM-Generated Network Changes? A Confirmatory Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a sealed paired confirmatory study (n = 200 intent→configuration tasks) to test whether a deterministic generate–validate–repair (GVR) pipeline that consults a deterministic NetOps validator reduces validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline. The run used a single hosted model (gpt-5-mini) with adapter-enforced shared-initial generation semantics and a primary deterministic validator (deterministic_netops_validator_v5). Execution completed all planned pairs (completed_pairs = 200) consuming 200 model calls (one shared initial generation per task); no repair calls were issued because every initial candidate passed the validator. Paired contingency: n11 = 200, n10 = 0, n01 = 0, n00 = 0; estimate = 0.0; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.0, 0.0] (10,000 resamples, seed = 1729). Because the baseline validator-detected unsafe incidence was 0% on this task bank, the preregistered >=50% relative-reduction hypothesis could not be meaningfully evaluated. We report the sealed design, execution accounting (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z), the Mininet 10% holdout (20 tasks) that produced zero validator–emulator disagreements, and an artifact-grounded discussion of operational implications and threats to validity (preregistration id: cnsms2026-gvr-effectiveness-02).**

I. Introduction

Automating intent→configuration translation with generative language models can increase NetOps productivity but risks producing incorrect or unsafe changes that cause outages or policy violations. Prior work therefore proposes grounding, deterministic validation, and repair loops (generate–validate–repair, GVR) to detect and correct unsafe candidates before application. The operational benefit of automated repair is conditional on three factors: (i) baseline prevalence of validator-detectable failures from the chosen generator/prompting policy; (ii) validator coverage and fidelity; and (iii) the available repair budget and repair-prompt effectiveness. We preregistered a narrowly scoped confirmatory question: Does an automated, deterministic GVR pipeline that consults a deterministic NetOps validator materially reduce validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline on a curated 200-task intent→configuration bank? The study was designed as a paired experiment that fixes the generator output per task and isolates the marginal effect of invoking a validator+repair on the same candidate.

II. Related Work

LLM failure modes, mitigation primitives, and NetOps prototypes. Systematic surveys and empirical studies across LLM applications have repeatedly documented hallucinations and factual errors, substantial prompt sensitivity, and non-determinism in generated outputs; these phenomena motivate a family of mitigations including retrieval grounding, verifier-assisted repair, and constrained or guarded agent architectures [10.1007/s11704-026-60308-3, https://openalex.org/W4399803256, https://openalex.org/W4402860127]. In the NetOps domain specifically, multiple prototype systems demonstrate that LLMs can translate natural-language intent into configuration fragments or workflows, but these works commonly stop short of instrumented, production-style failure-rate studies that quantify end-to-end operational risk [https://openalex.org/W4399629579, https://openalex.org/W4403635865].

Generate–Validate–Repair (GVR) and deterministic validators. Architectural proposals and early implementations advocate integrating LLM generation with deterministic, rule-based validators (Batfish-style or intent/constraint checkers) to prevent straightforward syntactic and semantic violations before application; GVR loops then attempt automated repairs when validators report violations [https://openalex.org/W4410125989, doi:10.36227/techrxiv.177004277.71795055/v1]. These integrations aim to convert an open-ended generative problem into a closed-loop verification problem where correctness is operationalized by validator predicates (for example: required_command_count, parsed_command_count, violation_codes). The literature also notes that validator coverage is crucial: deterministic checkers can be strong for the class of constraints they encode (topology-level invariants, required sequences), but may omit device-level idiosyncrasies or data-plane effects only visible under emulation or on real hardware [https://openalex.org/W4399629579]. That gap motivates the use of secondary end-to-end emulators or sampled real-device checks as contamination detectors in empirical evaluations.

Agentic workflows, tool-use brittleness, and red-team findings. Studies of agentic LLM systems and tool-using agents highlight new failure modes: tool-description or tool-registry

injection, cross-tool ambiguity, and persona/history interactions that change how agents interpret verifier responses or construct repairs [10.21203/rs.3.rs-10646209/v1, 10.2139/ssrn.7203631]. Red-team and audit reports show that apparently secure tool-assisted pipelines can be manipulated by adversarially crafted tool definitions or chained actions, producing dangerous configuration changes even when individual components seem benign. These findings imply that GVR effectiveness depends not only on validator logic but on the larger tool and agent context in which repair prompts and validator outputs are framed.

Evaluation gaps and the external-validation problem. Recent meta-analyses and reviews emphasize a recurring external-evaluation gap: many NetOps prototypes and LLM evaluations use synthetic or narrowly curated tasks and do not measure how validator-detected safety maps to real end-to-end risk across vendor devices, multi-vendor behaviors, or adversarial inputs [https://openalex.org/W4411523078, https://openalex.org/W4411735779]. That gap motivates three experimental priorities: (1) instrumented paired studies that fix generator sampling to isolate validator/repair effects, (2) inclusion of emulator or real-device holdouts to detect validator blind spots, and (3) reporting of repair budgets and model-call costs so practitioners can assess operational tradeoffs.

Prompt sensitivity, sampling semantics, and experimental design implications. Empirical work on prompt sensitivity and generation non-determinism shows that sampling policy (temperature, number of samples) and prompt templates materially affect observed correctness rates and the opportunities available to a repair loop [https://openalex.org/W4402860127]. Consequently, a paired shared-initial design—where the same initial candidate is reused across baseline and guarded arms—removes sampling variance and isolates the marginal effect of validation+repair on a fixed generator output. This design choice trades generality for controlled inference and is appropriate when the research question targets the marginal utility of repair for fixed generator behavior.

Synthesis and positioning. The prior work corpus collectively supports GVR as a promising architectural pattern but leaves unresolved empirical questions about when deterministic repair provides measurable operational benefit versus when efforts should focus on improving validator fidelity, diversifying generator sampling, or hardening agents against tool-description attacks. Our preregistered paired study was designed to address this specific empirical gap by fixing generator outputs (shared-initial semantics), enforcing a single-repair budget, and instrumenting a randomized emulator holdout as a contamination detector; the study therefore complements prior prototype and audit work by producing an externally auditable, artifact-grounded datum about GVR's marginal effect under a constrained but clearly specified operational regime.

III. Methodology

Preregistration and inferential plan. The study preregistration (cnsms2026-gvr-effectiveness-02) fixed a paired design with n = 200 tasks, one shared deterministic initial generation per task, and a repair budget of at most one automated repair call per task. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline; the primary test was an exact two-sided McNemar test ($\alpha = 0.05$). Confidence intervals were computed via bootstrap percentile (10,000 resamples, bootstrap_seed = 1729). Power planning assumed a baseline unsafe incidence $\approx$30% and a targeted 50% relative reduction; these assumptions were recorded in the preregistration and used only to justify sample size. The analysis executor was paired_binary_analysis_v1.

Task bank, validators, and holdout. We used a curated, machine-checkable intent$\rightarrow$configuration task bank with 200 tasks across four topology clusters (enterprise, campus, datacenter, edge; 50 tasks each). Each task encodes an initial and expected final state, workflow constraints (make-before-break, must_be_down_for_fields, group-active minima), and a required_sequence. The primary deterministic checker (deterministic_netops_validator_v5) enforces syntactic and intent/constraint checks and emits structured diagnostics (valid flag, violation codes, parsed_command_count, required_command_count). A Mininet-based data-plane emulator was preregistered as a secondary disagreement detector applied to a randomized 10% holdout (20 tasks) to detect validator–emulator mismatches; execution produced an analysis/contamination_summary.csv artifact recording zero disagreement flags for that holdout.

Model, adapter, and shared-initial semantics. The single hosted model used was gpt-5-mini (provider label: openai_responses) invoked via the hosted_netops_gvr_v1 adapter. Crucially, the adapter enforced shared_initial generation semantics: for each task the system produced a single initial candidate (deterministic sampling policy) which was deterministically reused for both baseline (LLM-only) and guarded (GVR) conditions. This paired, shared-initial design removes sampling variance across conditions and isolates the effect of validator+repair on a fixed generator output. Adapter accounting (deterministic_reconciliation artifacts) records hosted_model_call_event_count = 200 and stage_counts = {shared_initial_generation: 200}, confirming single shared invocations per task.

Repair policy, budgeting, and handling. The preregistered repair policy permitted one automated repair prompt to the same LLM when the primary validator reported violations, yielding a maximum of 200 repair calls across tasks. Missingness/exclusion rules deterministically excluded pairs where both conditions failed due to infrastructure execution failures and excluded tasks flagged for validator–emulator disagreement from the primary analysis per the contamination_plan; sensitivity analyses for alternative treatments were preregistered.

Execution accounting and artifact capture. The run executed autonomously between 2026-08-30T11:50:34.530059Z and 2026-08-30T12:12:07.541650Z under the frozen preregistration. The execution manifest and analysis artifacts (execution/raw_results.jsonl, analysis/paired_contingency_table.csv,

analysis/condition_summary.csv, analysis/contamination_summary.csv, analysis/analysis_log.jsonl, execution/model_configuration.json) were archived. All model calls, validator traces, and representative task artifacts (e.g., pair-000001...pair-000006) were recorded and are referenced in the analysis bundle.

IV. Results

Execution completeness and basic counts. The experiment executed to completion under the preregistered stopping rule. Accounting artifacts report completed_episode_count = 400 (shared initial generation reused across conditions), complete_pair_count = 200, failed_episode_count = 0, and model_calls_used = 200 (one shared initial generation per task). The adapter/provider audit recorded hosted_model_call_event_count = 200, cache_miss_count = 200, stage_counts = {shared_initial_generation: 200}, and condition_counts = {shared: 200}, confirming that no separate per-condition re-sampling occurred.

Validator outcomes and paired contingency. The primary deterministic validator (deterministic_netops_validator_v5) marked every shared initial candidate as valid. The condition summary artifact (analysis/condition_summary.csv) contains: "baseline,200,0,200,1.0" and "guarded,200,0,200,1.0" (success_rate = 1.0 for both conditions). The paired contingency artifact (analysis/paired_contingency_table.csv) records the contingency table used in the exact McNemar test as: - n11 (pass/pass) = 200 - n10 (guarded pass, baseline fail) = 0 - n01 (guarded fail, baseline pass) = 0 - n00 (fail/fail) = 0

Inferential result and bootstrap CI. With zero discordant pairs the observed paired difference estimate = 0.0. The pre-registered exact McNemar two-sided test returns p = 1.0. The bootstrap percentile 95% confidence interval computed with 10,000 resamples and bootstrap_seed = 1729 is [0.0, 0.0]. These numerical outputs are recorded in analysis/results.json and the analysis/analysis_log.jsonl artifact (bootstrap_resamples = 10000, bootstrap_seed = 1729).

Repair activity and secondary outcomes. The preregistered repair budget allowed up to 200 repair calls (one per task) but no repair calls were issued because no initial candidate failed the validator. The secondary_results array is therefore empty and the intended secondary estimands (average repair iterations, GVR-introduced failure rate, model-call cost per successful repair) are unpopulated in the analysis bundle. The analysis logs explicitly note repair_calls_used = 0.

Representative per-task diagnostics. Representative scoring artifacts for sample pairs illustrate validator traces and complexity indicators while still producing validator-passing configurations. Examples taken from archived representative tasks: - pair-000001: validator_trace.validation_after.valid = true; violation_count = 0; parsed_command_count = 4; required_command_count = 4; difficulty.level = "medium"; pattern = "shutdown_configure_restore"; required_sequence length = 4. - pair-000002: validator_trace.validation_after.valid = true; violation_count = 0; parsed_command_count = 2; required_command_count = 2; difficulty.level = "hard"; pattern = "make_before_break_uplink_switchover". - pair-000003: validator_trace.validation_after.valid = true; violation_count = 0; parsed_command_count = 6; required_command_count = 6; difficulty.level = "hard"; pattern = "paired_link_atomic_mtu_change". These artifacts demonstrate that tasks exercised medium-to-very_hard workflow constraints (examples in the archive include labels "medium", "hard", and "very_hard") and that the model outputs conformed to the task-encoded required_sequences and validator checks despite nontrivial required_command_counts.

Emulator (contamination) check. Per the preregistered contamination_plan, a Mininet-based data-plane emulator was run on a randomized 10% holdout (20 tasks). The analysis/contamination_summary.csv artifact records zero disagreement flags between deterministic_netops_validator_v5 and the Mininet emulator on that holdout (flag counts empty). Deterministic reconciliation artifacts further report pair_accounting_consistent = true and all deterministic consistency checks passed. We note, however, that the explicit labeled seed for the randomized holdout selection is not present among the labeled artifacts; the contamination_summary artifact nevertheless documents that the secondary emulator check executed and found no disagreements for the recorded holdout.

Unexecuted sensitivity branches and their artifact evidence. Several preregistered sensitivity analyses and contingency branches (e.g., treating flagged tasks as failures or successes, alternative missingness treatments, and inclusion of flagged tasks in the primary analysis) were not invoked because no tasks were flagged and no episodes failed. The execution manifest and analysis artifacts explicitly record these branches as unused; the analysis/contamination_summary.csv and analysis/missingness_summary.csv reflect zero flags and zero missing pairs.

Operationally relevant diagnostics. The run produces three operationally meaningful, artifact-grounded diagnostics: (1) baseline validator failure prevalence in this curated workload was 0% (200/200 passes) under the chosen generator/prompting policy and sampling semantics; (2) the adapter enforced shared_initial semantics eliminated per-condition sampling variance, so the absence of discordant pairs is not attributable to sampling differences across conditions but to the generator outputs themselves; and (3) the Mininet holdout found no validator–emulator mismatches on the 20-task sample, providing limited end-to-end corroboration of validator pass verdicts for that holdout. All of these points are directly supported by the archived artifacts referenced above (execution/model_configuration.json, deterministic_reconciliation.json, analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/contamination_summary.csv, and representative task scoring artifacts).

V. Discussion

Conditional interpretation. The degenerate null outcome demonstrates a logical constraint: when the cho-

sen generator, prompt template, and sampling policy produce validator-passing candidates for every item in a workload, an added validator-coupled repair step cannot reduce validator-detected failures. This observation is artifact-grounded and operationally important: the marginal value of automated repair depends on baseline validator failure prevalence and on validator coverage.

Why baseline failures were zero (artifact-based explanations). First, the prompt template and enforced shared_initial semantics produced outputs that conformed to explicit task-bank constraints. Second, the task bank is curated with clear required_sequences and machine-checkable constraints that map directly to validator checks—this reduces semantic ambiguity and makes correct outputs easier to generate with a deterministic prompt policy. Third, the primary validator enforces the task-encoded constraints but may omit device-level idiosyncrasies detectable only by an end-to-end emulator or real device; the recorded Mininet holdout produced zero disagreements but the run also discloses a missing labeled selection seed for that holdout (see reproducibility note). Fourth, the single repair budget and shared_initial design remove per-condition re-sampling as a pathway to obtain alternative repairable candidates.

Operational implications. Before deploying automated repair, practitioners should measure baseline validator failure prevalence on representative operational workloads and examine validator coverage with end-to-end emulation. When baseline validator failures are rare under the chosen generator/prompt policy, investing in richer validator fidelity (vendor semantics, device models), multi-sample generation, multi-model ensembles, or targeted adversarial testing may yield higher safety returns than automated repair. Conversely, when baseline failures are non-negligible, a well-engineered GVR pipeline with sufficient repair budget can reduce validator-detected issues; realized gains will depend on repair-prompt quality, budget, and validator completeness.

Threats to validity. Internal validity: preregistration, deterministic adapter enforcement, and frozen stopping rules minimize post-hoc choices, but the shared_initial design intentionally trades sampling generality for a controlled paired comparison. Construct validity: the primary validator enforces explicit task constraints but may not model all real device behaviors; emulator disagreements would reveal such blind spots but none were observed on the 20-task Mininet holdout. External validity: a single hosted model (gpt-5-mini), single prompt/sampling policy (shared initial deterministic sampling), a curated synthetic/public task bank, and a single repair budget were used; results may differ with other models, nonzero temperature sampling, multiple samples per condition, adversarial workloads, or production device heterogeneity. Conclusion validity: with zero discordant pairs the McNemar test is uninformative for the preregistered effect size; nonetheless the observed zero-discordance is a reproducible operational datum documented in the archived artifacts.

Reproducibility note and documented gap. All execution artifacts (model-call logs, validator traces, paired contingency tables, bootstrap parameters) are archived in the execution bundle. The Mininet holdout evidence is captured in analysis/contamination_summary.csv which reports zero disagreements for the 20-task holdout; this artifact demonstrates that the secondary emulator check executed and found no validator–emulator mismatches. A remaining reproducibility gap is that the explicit randomized selection seed for the Mininet holdout is not labeled in the archived artifacts, preventing exact deterministic reconstruction of which tasks were drawn into the holdout despite the contamination summary reporting zero disagreements. We disclose this gap transparently in the Disclosure Statement.

VI. Limitations

- Task-bank scope: the confirmatory sample used a curated synthetic/public intent→configuration bank (n = 200); results therefore reflect this bank and may not generalize to broader production workloads or adversarial distributions.
- Single-model and shared-initial setting: only gpt-5-mini (hosted) with adapter-enforced shared-initial generation was tested (execution artifacts record hosted_model_call_event_count = 200 and stage_counts shared_initial_generation = 200). Outcomes may differ across model families, independent per-condition sampling, nonzero temperature sampling, or larger repair budgets.
- Validator coverage and oracle limitations: the primary deterministic validator enforces a defined set of syntax/intent constraints; validators can fail to capture real device idiosyncrasies, vendor-specific behaviors, or emergent failure modes detectable only by end-to-end deployment.
- Adversarial robustness untested: this confirmatory run did not include active adversarial injection or red-team tool-description attacks; prior audits indicate such vectors can bypass naive defenses and remain a separate concern.
- Reproducibility gap for contamination check: the randomized holdout selection seed for the Mininet contamination check is absent from the labeled artifacts, preventing exact reconstruction of the holdout selection.
- Single repair budget: the GVR pipeline allowed at most one automated repair prompt per task; multi-step repair strategies, different repair budgets, or human-in-the-loop approvals were not exercised here.

VII. Conclusion

We preregistered and executed a sealed paired study comparing LLM-only generation to a deterministic GVR pipeline on 200 NetOps intent→configuration tasks using gpt-5-mini and deterministic_netops_validator_v5. Both pipelines produced validator-passing configurations for every task (n11 = 200; no discordant pairs), resulting in an observed paired difference of 0.0, exact McNemar p = 1.0, and bootstrap 95% CI = [0.0, 0.0]. The preregistered >=50% relative-reduction hypothesis could not be evaluated because baseline validator

failures were absent. The artifact-grounded outcome clarifies that GVR's marginal value is workload dependent: when baseline validator-detectable failures are rare, automated repair yields no measured benefit for those validator-detected errors. Future studies should target workloads with nontrivial baseline failure rates, expand validator fidelity with richer emulation or real-device checks, evaluate multi-sample and multi-model policies, and explore multi-step or human-in-the-loop repair budgets to characterize practical operational gains. All execution artifacts and analysis outputs are archived for independent inspection under the preregistration id cited above.

Disclosure Statement

Disclosure (mandatory). This study executed under the autonomous post-lock preregistration cnsms2026-gvr-effectiveness-02. Declared model: gpt-5-mini (provider label: openai_responses) invoked via the hosted_netops_gvr_v1 adapter. The exact initial master prompt is archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba). Topic constraints: the human Research Director limited the study to Generative AI and LLMs for NetOps focusing on safety/reliability of generated network changes. Frameworks and tools: orchestration used hosted_netops_gvr_v1 and the deterministic task generator deterministic_netops_task_generator_v5. Primary deterministic validator: deterministic_netops_validator_v5 (intent/constraint checks). A Mininet-based emulator was preregistered and executed as a secondary disagreement detector on a randomized 10% holdout (20 tasks); the artifact analysis/contamination_summary.csv shows zero disagreements for that holdout. Preregistration and freeze: the experimental plan (estimand, sample size n = 200, paired design, stopping rules, max model calls = 400, repair budget = 1 per task, shared-initial generation semantics) was frozen before any model calls. Autonomous execution and artifacts: the run executed autonomously under the frozen plan (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z); model calls, prompts, seeds, validator traces, emulator traces, logs, and the artifact manifest are archived in the execution bundle. Model-call budget and usage: 200 model calls were used (one shared initial generation per task); up to 200 repair calls were allowed but none were applied because initial candidates passed the validator. Human involvement: the human Research Director selected the topic family and pre-lock constraints; no human manual editing or selective rewriting of technical manuscript content occurred after the autonomy lock. Deviations and restarts: no post-preregistration design changes or technical restarts occurred. Known reproducibility gap: the explicit randomized selection seed used to draw the Mininet 10% holdout is not present as a labeled artifact in the archive; this absence prevents exact deterministic reconstruction of the drawn holdout despite the contamination summary reporting no disagreements. The master prompt archive reference above is the immutable prompt required by the track.

References

[1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

[2] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[3] Oscar G. Lira, Oscar Maurício Caicedo Rendón, Nelson L. S. da Fonseca, "Large Language Models for Zero Touch Network Configuration Management", 2024, 10.1109/mcom.001.2400368

[4] Chirag Shah, Ryen W. White, Reid Andersen, Georg Buscher, Scott Counts, Sarkar Snigdha Sarathi Das, Ali Montazeralghaem, Sathish Manivannan, J. Neville, Nagu Rangan, Tara Safavi, Siddharth Suri, Mengting Wan, Leijie Wang, Longqi Yang, "Using Large Language Models to Generate, Validate, and Apply User Intent Taxonomies", 2025, 10.1145/3732294

[5] omar altawil, Dr. Mustafa Al-Fayoumi, "Configuration-Level Semantic Brittleness in Tool-Using LLM Agents: A Factor-Ranking Study of Persona, History, and Tool Definition", 2026, 10.2139/ssrn.7203631

[6] Mohammadreza Rashidi, "Measuring How LLM Tool Descriptions, Cross-Tool Ambiguity, and Action Chains Compose into Excessive Agency Exploits", 2026, 10.21203/rs.3.rs-10646209/v1

[7] Yoshihiro Ando, "Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents", 2026, 10.36227/techrxiv.177006109.97957916/v1

[8] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[9] Shuyin Ouyang, Jie M. Zhang, Mark Harman, Meng Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation", 2024, 10.1145/3697010

[10] Oghenekeno Hilkiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1